WORKSHEET: NATURAL LANGUAGE PROCESSING

Name: _____ Branch/Year: _____ Date: _____

OBJECTIVE: Transform raw, messy human language into a clean, numerical format that a machine can understand, and finally build a Recurrent Neural Network (RNN) to classify it.

THE SCENARIO: You are a Junior AI Engineer at a tech startup. Your boss wants you to build a system that automatically flags "Spam" text messages. But first, you have to clean the dirty data.

THE RAW DATA

We will track **one single sentence** through this entire worksheet to see how it changes at every stage.

```
"Ugh... The deliveries were DELAYED!! I seriously hate
waiting... #annoyed"
```

PART 1: NOISE REMOVAL & NORMALIZATION

Computers are extremely literal. To a computer, "Apple", "apple", and "apple!" are three completely different words. This confuses the model. We need to standardize them.

🧠 Intuition (Do it by hand)

Task: Take the Input Sentence above. Convert everything to **lowercase** and **remove all punctuation** (dots, exclamation marks, hashtags).

Write your output here:

.....

.....

Reflection: Why do we remove punctuation?

[ ] It adds too much meaning.
[ ] It usually doesn't help determine the core meaning of words (e.g., "free!" vs "free").

💻 The Code

[ ] **Python Implementation**

```python
import string

text = "Ugh... The deliveries were DELAYED!! I seriously hate waiting... #annoyed"

# 1. Lowercase
text_lower = text.lower()

# 2. Remove Punctuation using string translation
# This creates a table that swaps punctuation for 'Nothing'
translator = str.maketrans('', '', string.punctuation)
clean_text = text_lower.translate(translator)

print("Original:", text)
print("Cleaned: ", clean_text)
```

PART 2: TOKENIZATION

Models don't read sentences; they read words (or "tokens"). We need to chop the stream of text into individual units.

🧠 Intuition (Do it by hand)

Task: Take your result from Part 1. Split it into a list of individual words.

Write your list here:

```
[ "________", "________", "________", "________", "________", "________",
"________", "________", "________" ]
```

Count: How many tokens do you have? .....

💻 Method A: The Simple Way (Split by Space)

[ ] **Python (Simple)**

```python
# The simple way (split by space)
tokens = clean_text.split()

print("Tokens:", tokens)
print("Token Count:", len(tokens))
```

🚀 Method B: The Sophisticated Way (BERT Subword Tokenization)

The Problem: What happens if the model sees a rare or made-up word? Modern models (like BERT/GPT) use **Subword Tokenization** to break complex words into known chunks.

Example Sentence: "The microtransactional system was counterintuitive."

[ ] **Python (Transformers)**

```python
# !pip install transformers
from transformers import AutoTokenizer

# We will use the BERT tokenizer (Standard in industry)
# This requires internet to download the vocab
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
complex_text = "The microtransactional system was counterintuitive."

# Output will likely split 'microtransactional' into ['micro', 'transaction', 'al']
print("Tokens:", tokenizer.tokenize(complex_text))
```

PART 3: STOP WORD REMOVAL

In NLP, words generally fall into two categories:

- **Content Words:** Nouns, verbs, adjectives (e.g., "Pizza", "Eat", "Tasty").
- **Stop Words:** Grammar words that provide structure but less meaning (e.g., "The", "is", "at").

🧠 Intuition (Do it by hand)

Sentence: "Ugh... The deliveries were DELAYED!! I seriously hate waiting... #annoyed"

Tokens: `['ugh', 'the', 'deliveries', 'were', 'delayed', 'i', 'seriously', 'hate', 'waiting', 'annoyed']`

Task: Cross out the Stop Words. Keep the Content Words.

💻 The Code

[ ] **Python (NLTK)**

```python
import nltk
from nltk.corpus import stopwords

# nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

filtered_tokens = [word for word in tokens if word not in stop_words]

print("Input:", tokens)
print("Result:", filtered_tokens)
```

🔴 A Critical "Food for Thought"

Consider this sentence: "*The movie was NOT good.*"

Question: In the sentence above, will you remove the word "not"?
Answer: .....

The Reality Check: If you blindly remove stop words using a standard library, "not" is often removed.
Result: "Movie good" (The meaning has flipped!)

> **Note:**
>
> - **Simpler Models (e.g., Naive Bayes, Logistic Regression):** We **MUST** remove stop words. These models run on low-end devices (like IoT sensors or basic phones) where memory is limited. We sacrifice a little context to make the model fast and small.
> - **Sophisticated Models (e.g., RNNs, Transformers):** We **KEEP** stop words. These models are smart enough to understand that "not" flips the meaning of the sentence.

PART 4: STEMMING VS. LEMMATIZATION

We have words like "deliveries" and "waiting". If we train a model on "waiting", and the user types "wait", the model won't know they are related. We need to cut words down to their **root form**.

- **Stemming:** A crude chopper. (Fast, but sometimes makes non-words).
- **Lemmatization:** A dictionary look-up. (Accurate).

🧠 Intuition (Do it by hand)

Task: Convert these words to their roots.

Original Word	Stemming (Chop)	Lemmatization (Dictionary)
deliveries	deliveri	delivery
waiting	wait	waiting
delayed	_____	_____
studies	_____	_____

💻 The Code

[ ] **Python (NLTK)**

```python
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer

nltk.download('wordnet')

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

words = ["deliveries", "waiting", "delayed", "studies"]

for w in words:
    # Notice: Stemmer makes "studi", Lemmatizer finds "study"
    print(w, "|", stemmer.stem(w), "|", lemmatizer.lemmatize(w))
```

PART 5: VECTORIZATION (THE "BAG OF WORDS")

The Starting Point: We still have text strings: `['ugh', 'delivery', 'delay', 'hate', 'wait', 'annoy']`.

Math equations (and RNNs) cannot multiply strings. We must convert them to numbers.

🛑 The Naive Approach

Idea: "Should we convert them directly to numbers? Just number them! Hate=1, Waiting=2, Annoy=3..."

Question: Why does this fail?

If Apple=1 and Banana=2 and Carrot=3...

Why this fails:

.....

.....

What's the solution: .....

⚠️ The Problem with One-Hot Encoding

If we use One-Hot Encoding, we need a new column for **every single unique word** in the English language.

If our vocabulary has 50,000 words, every single input vector will have 50,000 columns (mostly zeros).

This is huge, wasteful, and computationally impossible for large datasets.

✅ The Solution: Bag of Words (Frequency)

We assume that sentences containing the **same set of words** generally have the **same meaning**.

Example: "Good movie" and "Movie is good" share the same words, so they are similar.

Step 1: The Global Vocabulary
Our AI knows only these 10 words (The Universe):
`[ "win", "free", "money", "ugh", "deliveries", "delayed", "seriously", "hate", "waiting", "annoyed" ]`

Step 2: Demonstration
Let's vectorize a Spam sentence: **"Win free money"**
We check the list. If the word appears, we mark it.

win	free	money	ugh	deliveries	delayed	seriously	hate	waiting	annoyed
1	**1**	**1**	0	0	0	0	0	0	0

Step 3: Your Turn
Vectorize your preprocessed sentence:

```
['ugh', 'deliveries', 'delayed', 'seriously', 'hate', 'waiting', 'annoyed']
```

win	free	money	ugh	deliveries	delayed	seriously	hate	waiting	annoyed
—	—	—	—	—	—	—	—	—	—

PART 6: VECTOR EMBEDDINGS 🚀

The Limitation: The "Bag of Words" above has two big problems:

1. **It's Huge:** If English has 50,000 words, your vector has 49,998 zeros.
2. **It's Dumb:** It thinks "Happy" and "Joy" are completely different words because they have different slots.

The Solution: Word Embeddings.

🧠 Intuition (The DNA)

The computer learns a "Personality Vector" for each word. It assigns similar numbers to words with similar meanings.

Word	Roll No.	The Learned Embedding Vector
King	90	`[ 0.99, 0.99, 0.05 ]` (Royal, Male, Not food)
Queen	95	`[ 0.99, 0.05, 0.05 ]` (Royal, Female, Not food)
Apple	400	`[ 0.01, 0.10, 0.95 ]` (Not royal, Neutral, Food)

The Magic: Notice how King and Queen have almost the same numbers? The model automatically learned they are related!

💻 The Code (Just Watch)

In the real world, we just download a pre-trained brain (like a mini-ChatGPT) and ask it for the numbers.

[ ] **Python (Sentence Transformers)**

```python
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer

# 1. Download the Brain (Pre-trained on billions of sentences)
model = SentenceTransformer('all-MiniLM-L6-v2')

# 2. Give it words
words = ["King", "Queen", "Apple"]
embeddings = model.encode(words)

# 3. See the Vectors (The DNA)
for i in range(3):
    print("Word:", words[i])
    print("Vector:", embeddings[i][:3]) # Show just first 3 numbers
```

PART 7: PADDING

All sentences must be of the same length

Before we code the RNN, there is one final rule: **Inputs must be the same size.**

Imagine a classroom where every row must have exactly 10 desks.

- **Sentence A (Short):** "I love learning about AI." (5 words)
- **Sentence B (Long):** "The quick brown fox jumps over the lazy dog." (9 words)

We must **PAD** the short sentence with zeros to match the longest one.

Task: Pad Sentence A to match Sentence B (Target Length = 9).

Sentence A (Original): `[ 90, 400, 22, 10, 5 ]`
Sentence B (Original): `[ 50, 12, 88, 76, 45, 11, 23, 90, 100 ]`

```
Sentence A (Padded): [ 90, 400, 22, 10, 5, 0, 0, 0, 0 ]
Sentence B (Padded): [ 50, 12, 88, 76, 45, 11, 23, 90, 100 ]
```

(Note: The zeros act like empty desks. The model ignores them.)

THE GRAND FINALE: BUILDING YOUR FIRST RNN

Part 8: The "Toy" Dataset

We will use 6 longer sentences to prove our code works.

[ ] **Code Cell 8**

```python
import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, Embedding

# --- 1. SETUP DATA (The only part that changes) ---
train_sentences = [
    'The movie was truly amazing and exciting',     # Positive (1)
    'I absolutely loved the delicious food',        # Positive (1)
    'This is the best day ever',                    # Positive (1)
    'The service was terrible and very slow',       # Negative (0)
    'I really hated the boring movie',              # Negative (0)
    'This is the worst experience ever'             # Negative (0)
]
train_labels = np.array([1, 1, 1, 0, 0, 0])

# --- 2. THE PIPELINE (Standard for every NLP project) ---
# A. Tokenize & Integer Encode (Assign Roll Numbers)
# oov_token handles words we haven't seen before (Out Of Vocabulary)
tokenizer = Tokenizer(num_words=100, oov_token="<OOV>")
tokenizer.fit_on_texts(train_sentences)
sequences = tokenizer.texts_to_sequences(train_sentences)

# B. Padding (Make them all same length)
max_length = 8
padded_data = pad_sequences(sequences, maxlen=max_length, padding='post')

print("Word Index:", tokenizer.word_index)
print("Padded Data:\n", padded_data)

# --- 3. THE MODEL (The Brain) ---
model = Sequential()

# Layer 1: Embedding (The DNA Learner)
model.add(Embedding(input_dim=100, output_dim=8))

# Layer 2: SimpleRNN (The Sequence Reader)
model.add(SimpleRNN(16))

# Layer 3: Output (The Decision)
model.add(Dense(1, activation='sigmoid')) # 0 to 1

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

# --- 4. TRAIN ---
model.fit(padded_data, train_labels, epochs=50)

# --- 5. PREDICT ---
test_text = ["I loved the exciting movie", "The food was terrible"]
test_seq = tokenizer.texts_to_sequences(test_text)
test_pad = pad_sequences(test_seq, maxlen=max_length, padding='post')
```

```
print("\n--- TEST RESULTS ---")
preds = model.predict(test_pad)
print("Predictions:", preds)
```

PART 9: THE REAL WORLD CHALLENGE 🌍

Now, we do the **exact same thing** but with 5,574 text messages.

[ ] **Code Cell 9**

```
import pandas as pd

# --- 1. SETUP DATA (Real World) ---
url = "https://raw.githubusercontent.com/mohitgupta-omg/Kaggle-SMS-Spam-Collection-
Dataset-/master/spam.csv"
df = pd.read_csv(url, encoding='latin-1')

# Prepare the lists
train_sentences = df['v2'].astype(str).tolist()

# Convert labels to numbers (Spam=1, Not_Spam=0)
# 'ham' means valid email (Not Spam)
df['v1'] = df['v1'].map({'spam': 1, 'ham': 0})
train_labels = df['v1'].values

# --- 2. THE PIPELINE (Identical to Part 8!) ---
vocab_size = 5000
max_length = 20

# oov_token handles new words (Out Of Vocabulary)
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(train_sentences)
sequences = tokenizer.texts_to_sequences(train_sentences)
padded_data = pad_sequences(sequences, maxlen=max_length, padding='post')

# --- 3. THE MODEL (Identical to Part 8!) ---
model = Sequential()
model.add(Embedding(input_dim=vocab_size, output_dim=16))
model.add(SimpleRNN(32))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

# --- 4. TRAIN ---
history = model.fit(padded_data, train_labels, epochs=5, validation_split=0.2)

# --- 5. TEST IT LIVE ---
def predict_spam(message):
    seq = tokenizer.texts_to_sequences([message])
    pad = pad_sequences(seq, maxlen=max_length, padding='post')
    score = model.predict(pad)[0][0]

    print("Message:", message)
    print("Score:", score)
    if score > 0.5:
        print("🚨 SPAM")
    else:
        print("✅ Safe (Not Spam)")
```

```
predict_spam("Congratulations! You won a $1000 Walmart gift card. Call now.")
predict_spam("Hey, are we still meeting for lunch?")
```